

Aula 7 - Busca Binária e Algoritmo Guloso

Problema: Busca

Dado um vetor de n números inteiros v ordenado de forma crescente, e um inteiro $alvo$, queremos achar exatamente em que posição de v o $alvo$ está contido. Se o $alvo$ não existir em nenhum lugar do vetor, retornaremos o índice -1.

A abordagem mais básica para esse problema (que vocês já colocaram em prática algumas vezes) consiste simplesmente em comparar todos os elementos de v com $alvo$ de forma iterativa:

```
int v[n];
int alvo;

// tratamento de entradas

for(int i = 0; i < n; ++i){
    if(v[i] == alvo) cout << "Achamos! Posição: " << i << endl;
    else if(i == n-1) cout << "Não achamos :(";
}
```

Embora uma busca linear (como esse algoritmo é chamado) seja extremamente simples de implementar, e perfeitamente funcional, **em um vetor ordenado** é possível conseguir um resultado bem melhor de eficiência.

Usando a notação da aula passada, no pior caso (o caso em que o alvo tá no último elemento, ou nem tá no vetor pra começo de conversa) faríamos n operações para chegar a uma conclusão, logo, esse algoritmo é $O(n)$.

Ideia mais eficiente

Se um vetor está perfeitamente ordenado, conseguimos:

1. Chutar uma posição m aleatória no vetor
 - a. Se $v[m]$ é maior que o nosso alvo, sabemos que o alvo está antes de $v[m]$
 - b. Se $v[m]$ é menor que o nosso alvo, sabemos que o alvo está depois de $v[m]$
 - c. Se for exatamente igual ao nosso alvo, achamos a resposta certa
2. Daí, podemos chutar uma nova posição m dentro de um intervalo um pouco menor do vetor.
3. Repetimos a mesma ideia, sempre reduzindo o intervalo.

Por exemplo, seja v o vetor abaixo de tamanho n , e nosso alvo o número 91:

i	0	1	2	3	4	5	6
$v[i]$	12	13	14	56	65	67	69

Poderíamos tomar $m = 5$ como chute inicial. Assim, $v[m] < alvo$ e portanto sabemos que o alvo está necessariamente em uma posição entre 5 e $n-1$. Poderíamos prosseguir com um novo chute $m = 10$ e perceber que como agora $v[m] > alvo$, nosso alvo está em uma posição entre 5 e 10. Dessa maneira, conseguimos sempre reduzir o intervalo de busca fazendo menos comparações que em uma busca linear.

Solidificando a ideia

"Um chute aleatório" não é uma ideia muito *algoritmizável*. Podemos formalizar melhor essa parte pensando em **um chute sempre no meio do intervalo de busca**.

Se sabemos que nosso alvo está necessariamente entre as posições i e f , um bom chute para m seria $m = (i + f)/2$. Ou seja, no meio do intervalo.

Daí, adotando essa ideia de ter dois ponteiros, um para o início do intervalo de busca e um para o fim do intervalo de busca, o protótipo do nosso algoritmo fica mais ou menos assim:

1. Inicialmente, $i = 0$, $f = n-1$. (intervalo de busca é o vetor inteiro)
2. $m = (i + f)/2$ (calculamos um ponteiro para a posição do meio do intervalo)
 - a. Se $v[m] == \text{alvo}$ achamos!
 - b. Se $v[m] < \text{alvo}$ então nosso intervalo de busca passa a ser $i = m + 1$ e f .
(metade da direita do intervalo de busca)
Repetir algoritmo do começo
 - c. Se $v[m] > \text{alvo}$ então nosso intervalo de busca passa a ser i e $f = m - 1$.
(metade da esquerda do intervalo de busca)
Repetir algoritmo do começo

E, se em algum momento $i > f$ significa que o alvo não estava no vetor pra começo de conversa.

O algoritmo em C++ fica mais ou menos assim:

```
int i = 0, f = n-1;
while(i <= f){
    int m = (i + f)/2;

    if(v[m] > alvo) f = m-1;
    else if(v[m] < alvo) i = m+1;
    else{
        cout << "Achamos! Posição: " << m;
        return;
    }
}

cout << "Não achamos.";
```

Esse algoritmo tem uma ordem de complexidade muito melhor do que a busca linear. Dada uma entrada de tamanho n , no pior dos casos, dividimos o intervalo de busca por 2 até que o mesmo tenha tamanho 1. A complexidade do algoritmo é o número de divisões que conseguimos fazer antes que isso aconteça, ou seja, $O(\log_2 n)$. Como, por definição, nós ocultamos as constantes na notação big O, ficamos apenas com $O(\log n)$.

Isso significa que para uma entrada de tamanho $n = 10^9$, fazemos ≈ 30 operações. Absurdamente bom.

Existe uma prova matemática que diz que, problemas de busca são limitados por $O(\log n)$, ou seja, a melhor complexidade que conseguimos para um problema de busca é de fato logarítmica. Em outras palavras, esse é o melhor algoritmo para esse tipo de problema.

OBS.: Esse algoritmo só funciona se o vetor estiver PREVIAMENTE ORDENADO.

Problema: Troco

Dada uma quantia inteira X em dinheiro, queremos o número **mínimo** de cédulas que precisamos para representar esse número. Assuma que as cédulas que temos disponíveis são infinitas e têm os seguintes valores: (100, 50, 20, 5, 1).

A ideia mais intuitiva pra esse problema é a seguinte: dividimos X por 100, e daí temos o número de cédulas de 100 que podemos usar. Depois dividimos o resto da divisão de X por 100, por 50, e temos o número de cédulas de 50 que podemos usar. E assim sucessivamente. Ou seja:

```

int n100, n50, n20, n5, n1;
int X;
int total_cedulas;

n100 = X/100;
n50 = (X%100)/50;
n20 = ((X%100)%50)/20;
...

total_cedulas = n100 + n50 + n20 + n5 + n1;

```

Lembrando que uma divisão armazenada em um inteiro é equivalente a “pegar apenas a parte inteira” (ou a divisão inteira em Python).

Esse algoritmo, embora pareça burrinho, é o algoritmo mais eficiente para esse problema. Esse é o exemplo mais básico possível de um **Algoritmo Guloso**. Essa é uma técnica de resolução de problemas que consiste na filosofia de, em uma determinada etapa do processo de resolução, eu sempre faço a coisa que me parece ser a melhor possível **naquele momento**, sem me preocupar exatamente com o futuro.

É exatamente o que fazemos aqui, nós não nos preocupamos com o futuro da resolução, a cada momento, nós só tentamos pegar a cédula de maior valor possível para representar nosso número, quando essa cédula não é suficiente, vamos para a próxima de maior valor possível.

Algoritmos gulosos têm a tendência a serem extremamente fáceis de programar, e bem difíceis de provar que estão corretos. Eles sempre parecem estar errados ou parecem burros à primeira vista.

Problema: Atendimentos

Imagine que você é o secretário de um dentista. Seu colega secretário é muito desleixado, ele marca os atendimentos sem verificar se os atendimentos resultam em uma colisão de horários. Você, ao notar o desastre, decidiu fazer um programa que, dado uma lista com todos os atendimentos, você determina o maior número possível de atendimentos que o dentista conseguirá de fato realizar. Isto é, o maior número de atendimentos possível que não colidem uns com os outros.

Considere um atendimento como um par (horário de início, horário de fim).

Notem que, para esse problema, não importa exatamente “quais” atendimentos você pega, desde que você pegue o maior número possível de atendimentos.

Vamos tomar como exemplo a seguinte lista de atendimentos:

- 10h - 12h → A
- 10h - 13h → B
- 9h - 13h → C
- 8h - 10h → D
- 7h - 10h → E
- 9h - 14h → F
- 14h - 19h → G
- 13h - 14h → H

Notem que a solução ótima são os conjuntos de atendimentos (D ou E; A ou B; H; G), isto é, temos múltiplas soluções ótimas (soluções que pegam o número máximo de atendimentos sem colisões entre si). E nossa resposta é o número de atendimentos 4.

Agora, como chegamos nessa solução ótima algoritmicamente? Spoiler: a solução é gulosa também.

A ideia é, a cada momento da execução do algoritmo, pegamos sempre o atendimento que **termina mais cedo** (e que não colide com nenhum atendimento que já pegamos). Isso parte do princípio que o atendimento que termina mais cedo é o que nos deixa com mais espaço na agenda para colocar mais atendimentos.

Isso parece contraintuitivo, a princípio, pensamos sempre em pegar o “atendimento mais curto” ou sempre pegar o “atendimento que começa mais cedo”.

A implementação desse algoritmo é complicada, temos algumas opções, mas a melhor usa algumas ferramentas das quais ainda não conhecemos.

Structs

Em C++, conseguimos **definir structs**, que nada mais são do que nossos próprios “tipos de dados” mais complexos que os tipos primitivos.

Por exemplo, podemos definir uma struct Pessoa como uma estrutura que possui os dados “nome”, “idade” e “altura”. Em C++, isso fica assim:

```
#include<bits/stdc++.h>
using namespace std;

struct pessoa{ // note que definimos as structs ANTES da main
    string nome;
    int idade;
    float altura;
}; // ponto e virgula obrigatório

int main(){
    // agora podemos definir pessoas livremente
    pessoa matias;
    matias.nome = "Gabriel Matias";
    matias.idade = 19;
    matias.altura = 1.88;

    pessoa bento;
    bento.nome = "Lucas Bento";
    bento.idade = 18;
    ...

    // e podemos usar os dados da mesma forma que variáveis normais
    pessoa mais_novo;
    if(matias.idade < bento.idade) mais_novo = matias;
    else mais_novo = bento;

    vector<pessoa> pessoas; // vetor de pessoas
}
```

Sort

Em aulas passadas, discutimos o problema de ordenar um vetor de inteiros. Nós, meros mortais, conseguimos dois algoritmos $O(n^2)$ feitos do zero por nós mesmos. Mas, felizmente, no C++ já possuímos uma função pré implementada `sort()` que ordena um vetor em $O(n \log n)$. Para usá-la, precisamos incluir a biblioteca `algorithm`:

```
#include <iostream>
#include <vector>
```

```
#include <algorithm> // já inclusa no bits/stdc++.h
// é a biblioteca que contém o sort
```

A função em si funciona da seguinte forma, dado um vetor qualquer de inteiros, digamos `v`:

```
sort(v.begin(), v.end()); // ordena de forma crescente o vetor
```

Possuímos, ainda, um terceiro parâmetro opcional para essa função, que recebe uma outra **função de comparação** que você mesmo pode customizar. Por exemplo, se tivermos a função:

```
bool compara(int a, int b){
    return a > b; // mesma coisa que "retorna true se a>b e false caso contrário"
}
```

Podemos usar, agora:

```
sort(v.begin(), v.end(), compara); // isso vai ordenar de forma decrescente o vetor
```

Combinando structs e sort

No nosso problema de atendimentos, podemos usar a seguinte abordagem:

Criar uma estrutura adequada

```
struct atendimento{
    int inicio;
    int fim;
}
```

Criar uma função auxiliar para ordenar os atendimentos por horário de fim

```
bool compara(atendimento a, atendimento b){
    return a.fim < b.fim;
}
```

Ordenar um vetor usando essa função

```
vector<atendimento> atendimentos;
sort(atendimentos.begin(), atendimentos.end(), compara);
```

Percorrer o vetor verificando se há colisões

```
int resposta = 1;
atendimento ultimo_atendimento = vetor[i];

for(int i = 1; i<atendimentos.size(); ++i){
    if(atendimentos[i].inicio > ultimo_atendimento.fim){
        resposta++;
        ultimo_atendimento = atendimentos[i];
    }
}
```

```
cout << resposta;
```

Esses passos simulam a ideia de “pegar sempre o atendimento com o menor valor de término e que não colide com os já pegados”, e configura um algoritmo guloso mais complexo do que o das moedas.